INTR 5100 Traffic Flow Theory

Lab Session III Guidelines

Look-Behind Adaptive Cruise Control

---

1 Objective

Lab Session III focuses on controller design, physical testing, and trajectory analysis using the configuration

$$\text{Leading vehicle } (0) \longrightarrow \text{Controlled vehicle } (1) \longrightarrow \text{Following vehicle } (2). \tag{1}$$

The leader follows a predefined speed-command pattern, and the following vehicle uses a predefined IDM. Each group designs at least one look-behind controller for vehicle 1 using information from vehicles 0 and 2 and its own state.

2 Preparation and Submission Before the Lab

Each group must submit `student_controller3.py` to the TA before Lab Session III begins.

- Modify only `student_controller3.py` in the supplied experiment program. Keep the function name and arguments unchanged. All controllers will be tested in the same software environment and under the same experimental settings.
- Groups are encouraged to design two or more controllers for comparison.

For the lab environment, refer to the `Lab_environment_codes` folder on Canvas. It contains the controller template, experiment code, and route map for Experiment III. Controller execution is defined in `three_vehicle_experiment.py`. The following-vehicle IDM is implemented in `idm_controller.py`. These supporting files are provided for reference and must not be modified.

2.1 Controller Design and Implementation

Implement the controller in `student_control`, following this structure:

```
def student_control(
    leader_state, student_state, following_state, history=None
) -> float:
    # Replace ... with your controller calculation.
    v_des = ...
    return float(v_des)
```

The arguments `leader_state`, `student_state`, and `following_state` contain the latest received states of vehicles 0, 1, and 2, respectively. Each provides `velocity`, `x`, `y`, and `heading`.

The program supplies `history` as a list of trajectory records accumulated since the start of the current run. Each record is a dictionary containing `role`, `t_from_start` (time in seconds), `x`, `y`, `heading`, `velocity`, and longitudinal route coordinate `s`.

The program selects the three longest straight sections for control. It calls `student_control` only when vehicles 0 and 1 are in the same selected section. The following-vehicle IDM operates when vehicles 1 and 2 are in the same selected section. When either condition is not met, the corresponding vehicle uses the current leader speed as its desired speed command.

The function returns `v_des`, the desired speed command for vehicle 1. If your controller calculates acceleration, convert it into a speed command inside the function using a control time step Δt_k in seconds. The default control period is 0.15 s.

3 During the Lab Session

1. The TA loads the submitted controller and runs `ta_experiment3_gui.py`.
2. Each controller is tested in the same environment.
3. The three vehicle trajectories and control records are collected and provided to the group after each run.

4 After the Lab Session

Each group should include the following in the final project report:

- controller design and objectives;
- analysis of the three vehicles' trajectories;
- evaluation of speed-disturbance propagation, spacing control, and safety using suitable quantitative indicators;
- comparison of controller performance, if applicable.

After each test, the group receives the corresponding run folder, including `trajectory.csv`, `commands.csv`, `leader_events.csv`, and `metadata.json`.